Day 14 – Combination + Subset Sum

1️⃣ Combination vs Permutation (Quick Revision)

Feature	Combination	Permutation
Order matter	❌ No	✅ Yes
Example	AB = BA	AB ≠ BA
Count	nCr	n!

2️⃣ Print All Combinations (String)

💡 Concept

👉 Index based selection
👉 Same element dobara use nahi karte

✅ Code

```java
package Day_14;

public class Combinations {

    public static void combine(String str, int index, String curr){

        if(curr.length() > 0){
            System.out.println(curr);
        }

        for(int i = index; i < str.length(); i++){

            // choose
            curr = curr + str.charAt(i);

            // explore
            combine(str, i + 1, curr);

            // backtrack (undo)
            curr = curr.substring(0, curr.length() - 1);
        }
    }

    public static void main(String[] args) {
        combine("abc", 0, "");
    }
}
```

🔥 Dry Thinking

str = "abc"

→ a
  → ab
  → ac
→ b

→ bc
→ c

3️⃣ Subset Sum Problem (Important 🔥)

💡 Problem

👉 Given array, find all subsets jinka sum = target

Example

arr = [1,2,1]
target = 2

Output:

[2]
[1,1]

✅ Code

```
package Day_14;

public class SubsetSum {

    public static void findSubsets(int[] arr, int index, int sum, int target, String curr){

        if(index == arr.length){

            if(sum == target){
                System.out.println(curr);
            }

            return;
        }

        // include
        findSubsets(arr, index + 1, sum + arr[index], target, curr + arr[index] + " ");

        // exclude
        findSubsets(arr, index + 1, sum, target, curr);
    }

    public static void main(String[] args) {

        int[] arr = {1,2,1};
        int target = 2;

        findSubsets(arr, 0, 0, target, "");
    }
}
```

4️⃣ Optimized Subset Sum (Pruning)

👉 Agar sum already target se bada ho gaya → stop

if(sum > target) return;

Updated Code

```
public static void findSubsets(int[] arr, int index, int sum, int target, String curr){

    if(sum > target) return;

    if(index == arr.length){
        if(sum == target){
            System.out.println(curr);
        }
        return;
    }

    findSubsets(arr, index + 1, sum + arr[index], target, curr + arr[index] + " ");
    findSubsets(arr, index + 1, sum, target, curr);
}
```

5️⃣ Combination Sum (Advanced 🔥)

👉 Same element multiple times use kar sakte hain

Code

```
package Day_14;

public class CombinationSum {

    public static void find(int[] arr, int index, int target, String curr){

        if(target == 0){
            System.out.println(curr);
            return;
        }

        if(target < 0 || index == arr.length) return;

        // include (same index again)
        find(arr, index, target - arr[index], curr + arr[index] + " ");

        // exclude
        find(arr, index + 1, target, curr);
    }

    public static void main(String[] args) {

        int[] arr = {2,3,6,7};
        int target = 7;

        find(arr, 0, target, "");
    }
}
```

6️⃣ Day 14 Revision Summary

- Core Patterns

1. Include / Exclude
2. For loop + backtracking
3. Pruning (important)

- Important Differences

Problem	Technique
Subset	include/exclude
Combination	loop + index
Permutation	swap